

Smart Contract Upgradability: A Structured and Natural Approach

1st Davut Çulha

Department of Software Engineering
Atilim University
Ankara, Turkey
davut.culha@atilim.edu.tr

2nd Ali Yazici

Department of Software Engineering
Atilim University
Ankara, Turkey
ali.yazici@atilim.edu.tr

Abstract— Software maintenance is crucial as technology rapidly evolves, requiring software to meet new demands and correct errors. Smart contracts, immutable programs on blockchains like Ethereum, face challenges despite their immutability, often needing updates for errors or new features. Smart contracts are upgraded using different patterns, which are not natural because most of them implement upgrades using low-level operations that deviate from their intended use. In other words, these patterns are not natural because upgrades are done by implementing workarounds. Moreover, smart contracts are also susceptible to security vulnerabilities because they may hold large amounts of money. In this paper, upgradability of smart contracts is considered a necessity. For this purpose, a more structured method is proposed by adding high-level features and combining inheritance properties of object-oriented languages. A key component of this method is the `gotoContract` variable, which allows for the redirection of function calls to upgraded contracts. The proposed method provides a complete upgrade of data and functions in smart contracts. It aims to minimize the effects of upgrades on end users of the smart contracts. Additionally, this natural way of upgrading will help mitigate security risks in the smart contracts by providing a high-level approach to upgrade.

Keywords—smart contract upgradability, upgradable smart contract, blockchain smart contract, smart contract maintenance, object-oriented inheritance, smart contract upgradability patterns

I. INTRODUCTION

Software maintenance is becoming increasingly important as the environment in which software operates continues to change rapidly. The rapid pace of technological advancement means that software must also evolve to meet new requirements and adapt to changing conditions. This evolution often takes the form of software updates and upgrades, which allow existing software to be modified to meet new needs and correct errors.

One area where software updates are particularly challenging is in the realm of smart contracts, which are blockchain-based programs that are designed to be immutable once deployed. Smart contracts, first introduced in the Ethereum blockchain, are designed to execute functions in a secure and transparent manner without the need for intermediaries. This paper proposes a natural way to upgrade smart contracts transparently.

Despite their immutable nature, smart contracts often require updates due to errors or the need for new features. Smart contracts may contain errors and bugs that need to be corrected to protect against potential vulnerabilities. As user requirements evolve, smart contracts may need updates to include additional functionalities. Moreover, smart contracts

may need updates to patch security vulnerabilities, ensure regulatory compliance, improve performance, and maintain integration and interoperability.

Updating smart contracts presents several challenges, including the need to ensure that all related software is updated to reflect the changes, the difficulty of managing the state of a contract with added data variables, and the challenge of managing authorization for updates. The state of smart contracts should be migrated to the new contracts for a seamless upgrade; otherwise, there would be incompatibilities between the old and new contracts. In addition, upgrades should not affect the users of the smart contracts as much as possible. Upgrading smart contracts can be thought of in two ways: data upgrade and function upgrade. In both cases, there should be the ability to change data and functions of smart contracts as much as possible. This paper proposes a method that allows for a complete upgrade of data and functions in smart contracts.

To address these challenges, this paper proposes a method for upgrading smart contracts that allows for complete upgrades of both data and functions. This method leverages high-level programming features and object-oriented inheritance properties to provide a more natural approach to upgrading. Central to this method is the `gotoContract` variable, an address variable that allows for the redirection of function calls to upgraded contracts. By setting this variable to the address of a new contract, the system can seamlessly direct calls to the upgraded contract, thus enabling a structured and secure upgrade path. This approach aims to provide a high-level, predefined method for upgrading smart contracts, minimizing disruptions for end users and enhancing security.

This paper explores the challenges and methods associated with upgrading smart contracts. Different types of maintenance can be applied to smart contracts, such as corrective, adaptive, perfective, and preventive maintenance. It explores various methods for upgrading smart contracts, including the use of proxy contracts and the deployment of new contracts to new addresses. The current upgrading patterns are not natural because upgrading is done using low-level operations of blockchains. This is a workaround against the immutability of smart contracts. However, upgrading smart contracts is necessary, and it should be achieved in a predefined and natural way. In this paper, a method is proposed to upgrade smart contracts naturally, rather than using a workaround.

The paper also discusses the importance of security in smart contract development and maintenance. Smart contracts are susceptible to security vulnerabilities, which can be exploited to steal funds or disrupt their operation. Various

security auditing methods and frameworks are discussed to mitigate these risks and ensure the secure deployment and maintenance of smart contracts. In this paper, a natural way of upgrading is proposed. This method will help mitigate the security risks in the smart contracts by providing a high-level upgrade method in a predefined way.

The contributions of this paper are manifold: it introduces the `gotoContract` variable as an innovative redirection mechanism, simplifying the upgrade process and enhancing security; it leverages object-oriented inheritance to provide a structured approach to smart contract upgrades, ensuring compatibility and minimizing user disruption.

Overall, this paper provides an overview of the challenges and methods associated with upgrading smart contracts, highlighting the importance of ensuring the security and integrity of smart contract deployments in the rapidly evolving world of blockchain technology. Moreover, this paper proposes a natural upgrade path for smart contracts.

The structure of the paper is as follows: related work is presented in the next section, followed by an explanation of the proposed method. After discussion and future work, the paper ends with a conclusion.

II. RELATED WORK

Software maintenance becomes more important day after day. The main reason for this is the changing environment. The world changes very fast as new technologies are found. One reflection of this is that software in the changed World also needs changes. Moreover, instead of developing completely new software, existing software is updated for already existing requirements and new requirements. On the other hand, there is software that is immutable. These are called smart contracts, which are blockchain programs and first defined in the blockchain Ethereum [1]. Smart contracts once deployed cannot be altered. However, in practice, smart contracts are updated because of several reasons. In [2] [3], smart contract maintenance is examined. The conventional maintenance types, which are corrective, adaptive, perfective, and preventive, and their applicability to smart contracts are analyzed. Moreover, methods related to maintenance of smart contracts are explored. In [4], a comprehensive survey is provided for smart contract upgradability. Many upgradability patterns are collected, and their features are studied to form a thorough catalog.

Smart contracts need upgrades because they may have errors and they may need additional features [2] [5]. If a smart contract has bugs, they should be fixed as upgrades. In addition, new features can be added to smart contracts as upgrades. Smart contracts publish their bytecode and balance to everyone. Therefore, they are open to security threats, and actions to be taken may be needed against possible security threats. Otherwise, millions of dollars may be stolen from smart contract balances.

Security issues in smart contracts are very important because smart contracts may hold large amounts of money. In [6] [7], security vulnerabilities in smart contracts are presented. Attack methods against security vulnerabilities are analyzed. Corresponding defense methods are presented. In [8], a framework is proposed to deploy smart contracts safely. The deployer framework verifies smart contracts before deployment and upgrades. The framework guarantees safe execution of smart contracts by the participants. In [9], a

secure framework is reviewed for the upgradability of smart contracts. The framework defends against attacks, and it offers a scalable upgradability pattern. In [10], a security analysis framework is designed to detect and analyze security issues of smart contracts. In addition, a taxonomy of upgradable smart contracts is developed. In [11], smart contract development is analyzed from the perspective of software engineering. Smart contract development has additional challenges because security and reliability issues in smart contracts have great importance. Therefore, special tools for development and debugging of smart contracts are needed for secure and reliable smart contract development. In [6], security audit of smart contracts is emphasized. A combination of static and dynamic auditing methods is proposed for various vulnerabilities. This paper proposes a smart contract upgrade method using high-level programming features, aiming to make upgrading smart contracts easier and more understandable. This approach would minimize the security risks in smart contracts by making the upgrading process straightforward.

Upgrading smart contracts has some challenges [12]. If upgrading is done by transferring the smart contract to a new address, then all the related software should update the address of the smart contract. If upgrading is done using a simple proxy mechanism, then addition of new functions to the smart contract would be problematic. On the other hand, state management of a smart contract becomes difficult because adding new data variables or using existing data is another challenge. In addition, how the authorization of upgrading is managed is another issue. If a single person or a single group has the upgrading authority, they may misuse their rights. Redeploying the new smart contract to the same address for upgrading can lead to incompatibilities between the old and new contract data and functions [13]. Upgrading a smart contract is difficult because there are currently not enough technical tools for upgrading smart contracts [11]. In [14], diamond upgrading pattern, which is a variation of proxy pattern, is described to overcome the challenges of upgradability. In this paper, the proposed method for upgrading smart contracts provides complete management on the data and functions of the smart contracts.

In [15], the challenges developers face during smart contract development are examined. First, there is no efficient method to assure the security of codes of smart contracts. Second, the capabilities of development tools are inadequate. Third, virtual machines and programming languages have some limitations. Fourth, achieving desired performance is impossible usually because resources in the blockchain networks are limited. Finally, there is not enough documentation and community support.

Smart contracts are Turing-complete programs which run on blockchain [2]. The first smart contracts were executed in Ethereum blockchain [1]. In Ethereum, the execution environment is a virtual machine called Ethereum Virtual Machine (EVM) which executes smart contracts [16]. Essentially, EVM executes bytecode which is the assembly code of Ethereum blockchain. Generally, smart contracts are written in high-level languages like Solidity, and Vyper. Then, these high-level programs are compiled into EVM bytecode and executed in EVM.

A big issue in upgrading smart contracts is related to administration of upgrading [12]. In other words, who will upgrade the smart contract is an issue, and this issue should be

addressed in accordance with the philosophy of blockchain. A smart contract can be upgraded by a single person who has the upgrade authorization. This method is not suitable with the decentralized nature of blockchain because the single person may misuse his/her permission, or the permission may be used by adversaries without his/her will [14]. The second way, a smart contract can be upgraded by a group of people. This is rather a good way unless the group is controlled by a single entity and the users of the smart contract are informed about this upgrade permission. The third way, a smart contract can be upgraded by a fair decentralized voting mechanism of the governance of the users of the smart contract. The last method may be the best method because it fits to the decentralized philosophy of blockchain [14] [17]. In this paper, the proposed method includes a function to determine who will upgrade the smart contract. This function helps users of the smart contract understand the upgrade process and administration of the smart contract. This approach aims to make the upgradability of the smart contract clearer to its users.

Smart contract upgrade patterns in Ethereum blockchain are examined in [18] [12] [14] [9] [10]. The first upgrade pattern is to deploy the smart contract to a new address [12]. The main disadvantage of this pattern is all the applications which use the smart contract have to change the address of the smart contract. Therefore, the upgrade will bring a lot of workloads. The second pattern is a simple proxy method, which automizes the new address operations. The proxy directs calls to the smart contract to a new contract address. The third upgrade pattern called data segregation pattern separates data from the logic of the contract [18] [19]. In this pattern, a more structured upgrade is realized in the logic part without touching the data part of the contract. However, data segregation pattern does not have upgrade capability in the data part. This pattern has some variations like basic data segregation, satellite pattern, register contract, and contract relay [18] [19]. The fourth pattern is proxy contract pattern [18] [4] [14]. Proxy contract pattern depends on the low-level operation DELEGATECALL in EVM. A smart contract executes functions in other contracts within the context of the caller contract using the DELEGATECALL opcode [18] [2] [4] [10]. Proxy contract pattern can be implemented with three different contracts which are a proxy contract, a logic contract, and a data contract. The proxy contract directs calls to the logic contract with the DELEGATECALL opcode, which uses data in the data contract. Proxy contract pattern has some variations like inherited proxy storage, eternal proxy storage, and unstructured proxy storage [18]. OpenZeppelin [20] is an open-source framework to write upgradable smart contracts on the Ethereum platform. Proxy contract pattern implementations are analyzed using OpenZeppelin upgradable smart contract patterns [21].

Upgradability of Ethereum smart contracts is investigated in [12]. Upgradability patterns are categorized as retail and wholesale changes. Retail changes mean not replacing the contract with a new one, only changing variables in the contract. Wholesale changes mean replacing the contract with new ones. Wholesale changes are divided into 4 groups. The first group has the pattern which creates a new contract at a new address [12]. The last three groups are formed according to the low-level operations in the EVM, which are CREATE2, CALL, and DELEGATECALL opcodes. CREATE2-based pattern creates a new contract at the old address [10]. In other words, the contract is replaced with a new one without changing the address of the contract. CALL-based pattern is

data segregation pattern. DELEGATECALL-based pattern is the proxy contract pattern. The most prominent pattern for upgrades is the proxy contract pattern [12]. In [13], proxy contract pattern is optimized by adding a new tier named verification layer, which verifies the security and correctness of the smart contract. Up to now, there is no efficient smart contract upgrade pattern [13]. Upgrading smart contracts using low-level operations is not natural, considering that the upgrading of smart contracts is inevitable. Therefore, this paper proposes a structured method and a natural way to upgrade smart contracts.

III. THE PROPOSED METHOD

Smart contracts will have an additional functionality to provide upgradability, which is realized through a special variable called gotoContract. The gotoContract variable is of type address, which holds the address of a smart contract.

The gotoContract address can be either initial or set to a specific address. A newly created smart contract will have an initial value for the gotoContract variable by default. Therefore, each call to the contract will be executed within the contract itself. However, if the gotoContract value is set to a new contract address, then each call to the contract will be executed in the contract referenced by the gotoContract address.

To securely set the gotoContract address, there should be a default mechanism (e.g., a built-in method). After setting the gotoContract address, each call to the contract will be directed to the new contract. The new contract will also have a gotoContract variable, which can be set later. This way, a chain of upgradable smart contracts can be built, where the last contract will be the only contract, and all calls will be directed to this last contract.

The gotoContract address will only be allowed to be set if the specified address points to an existing contract. This ensures that the possibility of an incorrect setting action is minimized.

The primary purpose of the gotoContract variable is to redirect function calls seamlessly to new versions of the smart contract without changing its external interface. By introducing gotoContract, smart contracts can achieve a level of flexibility that traditional approaches lack. Initially, when a smart contract is deployed, the gotoContract variable is set to an initial value. This indicates that all calls are to be handled by the deployed contract itself. However, when a new version of the contract is introduced, the gotoContract value can be updated to reference the new contract's address. As a result, all function calls are directed to the upgraded contract, enabling a smooth transition between different contract versions without altering user interactions or requiring manual intervention by external applications.

To ensure a robust implementation, several key enhancements to the concept of inheritance in smart contracts are proposed. The inheritance mechanism is redefined to introduce a more flexible approach that maintains a connection to the original contract's state without duplicating its data. This adjustment allows derived contracts to inherit functionalities while ensuring consistency in state variables across different versions, avoiding complications related to data migration. Additionally, the upgradability mechanism employs the gotoContract variable, which is managed through a controlled process to verify and authorize updates. This

additional layer of control prevents unauthorized or malicious changes, thereby enhancing the overall safety of the upgrade process.

```

contract A {
    address owner;
    constructor() {
        owner = msg.sender;
    }
    uint number = 0;
    function setNumber(uint newNumber) public {
        number = newNumber;
    }
    function getNumber() public view returns (uint) {
        return number;
    }
    function setGotoContract(address newContract) public {
        require(msg.sender == owner);
        this.gotoContract = newContract;
    }
}

contract B upgrades A {
    bool isNegative;
    function setNumber(uint newNumber) public {
        A.number = newNumber;
        isNegative = false;
    }
    function getNumber() public view returns (uint) {
        return A.number;
    }
    function setNumberAndIsNegative(uint newNumber, bool newIsNegative) public {
        A.number = newNumber;
        isNegative = newIsNegative;
    }
    function getNumberAndIsNegative() public view returns (uint, bool) {
        return (A.number, isNegative);
    }
    function setGotoContract(address newContract) public {
        require(msg.sender == A.owner);
        this.gotoContract = newContract;
    }
}

```

Fig. 1. Upgradable contract A and upgraded contract B examples in Solidity, depicted fictitiously.

To achieve fully upgradable smart contracts, additional programming language features need to be added to existing programming languages. For example, in Figure 1, fictitious smart contracts are shown in Solidity. Contract A is the initial smart contract deployed, and when a change to contract A is needed, a new smart contract B is created based on A using the inheritance mechanism in the proposed method.

The following features should be added to Solidity:

- In this proposal, the upgradability of smart contracts is based on inheritance. The inheritance mechanism for upgradability in the proposal has a subtle difference. Therefore, a new keyword is introduced: the keyword "upgrades" instead of the keyword "is" in Solidity. The keyword "upgrades" is similar to the keyword "is," but it does not keep a copy of the data of the base contract. Therefore, whenever there is a request regarding the base contract, the request will be executed in the base contract and will be accessible from the new contract. In other words, the new contract does not keep any data belonging to the base contract.
- A smart contract should have a special variable "gotoContract", which can be set externally for every smart contract. This variable holds the address value of another smart contract. Initially, this variable is set to empty ("address(0)").
- The special variable "gotoContract" should also have a default setter function like the following:

```

function setGotoContract(address newContract) public {
}

```

- The setter function "setGotoContract" can be defined by overriding the default function as follows:

```

function setGotoContract(address newContract) public {
    require(msg.sender == owner);
    this.gotoContract = newContract;
}

```

In this example, the variable can be set only by the creator of the contract. Therefore, a smart contract can be made upgradable only if the setter function "setGotoContract" is overridden by the developer of the contract.

- The setter function "setGotoContract" can be executed with a suitable address, which points to a contract that is deployed to the blockchain and is an inheritance of the upgradable contract. In this example, contract B is an inheritance of contract A. If contract B was deployed with an address, that address can be used to set the variable "gotoContract".
- In a smart contract, if the address "gotoContract" has been set, all the calls to the upgradable contract will be directed to the address of the new contract.
- The new contract can override all the elements of the upgradable contract. In this example, the function "setNumber" is overridden. It sets the number in the upgradable contract and also sets a new value "isNegative" in the current contract.
- If the new contract does not override an element, the calls will be executed in the upgradable contract.
- The upgradability of smart contracts can continue deeper and deeper for the derived smart contracts whenever the function "setGotoContract" is overridden in the derived smart contract. In this example, smart contract B is also upgradable because the function "setGotoContract" is overridden and it accepts changes from the creator of the root smart contract A.

Figure 2 shows the sequence diagram of the calls to the upgradable contract. If the gotoContract variable has not been initialized then the call is executed in the upgradable smart contract and returns. However, if the gotoContract variable is set to a new contract address, then the call is directed to the new contract and returns to the client.

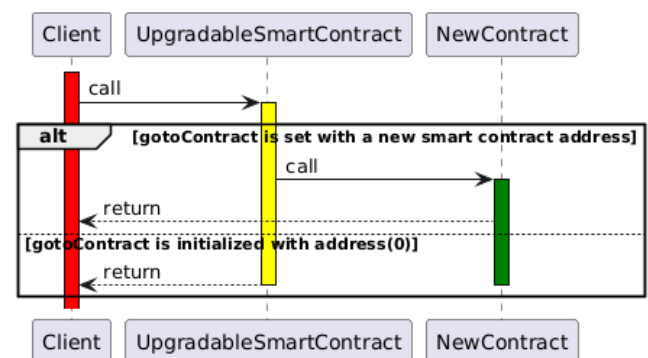


Fig. 2. Sequence diagram of calls to the upgradable contract

IV. DISCUSSION

The proposed method for upgrading smart contracts presents a significant advancement. By introducing the `gotoContract` variable and its associated functionality, the method provides a structured and natural approach to smart contract upgradability.

One key strength of this approach is its reliance on inheritance, a fundamental concept in object-oriented programming. By leveraging inheritance, developers can easily create new contracts that inherit from existing ones, allowing for seamless upgrades while maintaining compatibility with the original contract's functionality. This approach not only simplifies the upgrade process but also ensures that upgrades can be applied without disrupting the existing functionality of the contract.

Moreover, the use of the `gotoContract` variable adds an additional layer of flexibility to the upgrade process. By allowing the `gotoContract` address to be set externally, developers can easily direct contract calls to a new contract, enabling smooth transitions between different versions of the contract. This feature is particularly useful for contracts that require frequent updates or modifications.

To enhance the security and reliability of upgrades, a voting mechanism can be implemented to approve the upgrade. Only if the majority of users of the smart contract accept the upgrade, it can be executed.

Overall, the proposed method for upgrading smart contracts represents a significant step forward in smart contract development. By providing a structured and natural approach to upgradability, this method offers developers a powerful tool for maintaining and evolving their smart contracts in a secure and efficient manner.

V. FUTURE WORK

The proposed method can be implemented by changing the compiler of Solidity, leveraging its features to enable smart contract upgradability in a natural approach. Each smart contract should have the default variable `gotoContract`, and its associated setter function `setGotoContract`. The `gotoContract` variable is initialized to `address(0)` by default, indicating that calls should be executed within the contract itself. If the `gotoContract` variable is set to a new smart contract address, all the calls to the smart contract should be directed to the new smart contract. Moreover, the new smart contract should access the data and functions of the base smart contract if the `gotoContract` variable is set to the new smart contract address.

VI. CONCLUSION

The proposed method enables fully upgradable smart contracts, allowing both the data and function parts of a smart contract to be fully upgraded. The current upgrade patterns are not natural because most of them implement upgrades using low-level operations that deviate from their intended use. This approach provides a natural and structured method for upgrading by leveraging inheritance and allowing inherited smart contracts to control data and functions in the base contracts, offering flexibility and compatibility. Additionally, implementing a voting mechanism can enhance the security and reliability of upgrades.

The contributions of this paper include the introduction of the `gotoContract` variable, which simplifies the upgrade process and enhances security by providing a high-level

redirection mechanism. By leveraging object-oriented inheritance, the proposed method ensures compatibility and minimizes disruptions for end users during upgrades.

The method not only mitigates security risks but also provides developers with a powerful tool for maintaining and evolving their smart contracts. It allows for seamless upgrades while maintaining compatibility with existing functionality, simplifying the upgrade process, and minimizing disruptions for end-users.

Overall, this method offers a comprehensive solution to the challenges associated with smart contract upgradability, paving the way for more secure and efficient smart contract development and maintenance in the future.

REFERENCES

- [1] Wood G (2014) Ethereum: A secure decentralised generalised transaction ledger. Project Yellow Paper
- [2] Chen, J., Xia, X., Lo, D., Grundy, J., & Yang, X. (2021). Maintenance-related concerns for post-deployed Ethereum smart contract development: issues, techniques, and future challenges. *Empirical Software Engineering*, 26(6), 117.
- [3] Wöhrer, M., & Zdun, U. (2021, December). Devops for ethereum blockchain smart contracts. In *2021 IEEE International Conference on Blockchain (Blockchain)* (pp. 244-251). IEEE.
- [4] Meisami, S., & Bodell III, W. E. (2023). A Comprehensive Survey of Upgradeable Smart Contract Patterns. *arXiv preprint arXiv:2304.03405*.
- [5] Qasse, I., Hamdaqa, M., & Jónsson, B. Þ. (2023). Smart contract upgradeability on the Ethereum blockchain platform: An exploratory study. *arXiv preprint arXiv:2304.06568*.
- [6] He, D., Deng, Z., Zhang, Y., Chan, S., Cheng, Y., & Guizani, N. (2020). Smart contract vulnerability analysis and security audit. *IEEE Network*, 34(5), 276-282.
- [7] Qian, P., Liu, Z., He, Q., Huang, B., Tian, D., & Wang, X. (2022). Smart contract vulnerability detection technique: A survey. *arXiv preprint arXiv:2209.05872*.
- [8] Antonino, P., Ferreira, J., Sampaio, A., & Roscoe, A. W. (2022, September). Specification is Law: Safe Creation and Upgrade of Ethereum Smart Contracts. In *International Conference on Software Engineering and Formal Methods* (pp. 227-243). Cham: Springer International Publishing.
- [9] Girish, N. M., & Kaganurmath, S. (2022). Upgradability of smart contracts: A Review.
- [10] Bodell III, W. E., Meisami, S., & Duan, Y. (2023). Proxy hunting: Understanding and characterizing proxy-based upgradeable smart contracts in blockchains. In *32nd USENIX Security Symposium (USENIX Security 23)* (pp. 1829-1846).
- [11] Bosu A, Iqbal A, Shahriyar R, Chakraborty P (2019) Understanding the motivations, challenges and needs of Blockchain software developers: a survey. *Empirical Software Engineering* 24(4):2636{2673
- [12] Salehi, M., Clark, J., & Mannan, M. (2022). Not so immutable: Upgradeability of smart contracts on ethereum. *arXiv preprint arXiv:2206.00716*.
- [13] Du, Z., Cheng, H., Fu, Y., Huang, M., Liu, L., & Ma, Y. (2023). A Four-Tier Smart Contract Model with On-Chain Upgrade. *Security and Communication Networks*, 2023.
- [14] van Vulpen, P., Heijnen, H., Kroon, T., Mens, S., & Jansen, S. (2023). Decentralized Autonomous Organization Governance By Upgradeable Diamond Smart Contracts. Available at SSRN 4634762.
- [15] Zou, W., Lo, D., Kochhar, P. S., Le, X. B. D., Xia, X., Feng, Y., ... & Xu, B. (2019). Smart contract development: Challenges and opportunities. *IEEE Transactions on Software Engineering*, 47(10), 2084-2106.
- [16] Tara, A., Ivkushkin, K., Butean, A., & Turesson, H. (2019). The evolution of blockchain virtual machine architecture towards an enterprise usage perspective. In *Software Engineering Methods in Intelligent Algorithms: Proceedings of 8th Computer Science On-line Conference 2019*, Vol. 1 8 (pp. 370-379). Springer International Publishing.

- [17] van Vulpen, P., Siu, J., & Jansen, S. (2023). Governance of decentralized autonomous organizations that produce open source software. *Blockchain: Research and Applications*, 100166.
- [18] Bui, V. C., Wen, S., Yu, J., Xia, X., Haghighi, M. S., & Xiang, Y. (2021, December). Evaluating upgradable smart contract. In *2021 IEEE International Conference on Blockchain (Blockchain)* (pp. 252-256). IEEE.
- [19] M. Wöhrer and U. Zdun, "Design patterns for smart contracts in the ethereum ecosystem," 2018.
- [20] "Proxies," OpenZeppelin Docs. [Online]. Available: <https://docs.openzeppelin.com/contracts/4.x/api/proxy>.
- [21] Amri, S. A., Aniello, L., & Sassone, V. (2023). A Review of Upgradeable Smart Contract Patterns based on OpenZeppelin Technique. *The Journal of The British Blockchain Association*.